Agent Hub 演讲稿

第 1 页｜开场

大家好，今天我想站在 Designer 的角度，分享我们如何从一个产品问题出发，和 AI 一起，把初步想法逐步拆解成角色、流程和可以继续维护的项目文件，让整个项目变得清晰、可审查，也能持续迭代。

第 2 页｜沿着九月份的路线继续

九月份的分享，我们讲的是怎样从 Chat 到 Prompt，再到 Context，一步步让 AI 任务变得可靠。也就是把目标、输入、规则和交付说清楚，再通过 Evidence 来核对结果。那次主要关注的是，怎样把一次 Task 设计好。

这次呢，我们从 Designer 的角度再往前想一步。如果这些任务反复出现，我们应该怎样提高效率，或者搭建 Agent 呢？这就进入了后面的 Coordinated Work。我们需要用 Workflow 把步骤连起来，让 Agent 明确各自的职责和范围。简单来说，就是回答这个问题，谁做什么、交接什么、在哪里停。

今天我们会用 Agent Hub 这个 case，来帮助大家理解接下来要讲的内容。这条线帮助我们理解两次分享之间的联系，也就是从做好一次任务，到让多个角色协作起来。

第 3 页｜先看从 Chat 到 Agent Hub 的整体路线

我们先把今天的内容放到一张图里看。先把需求说清楚，再把设计保存成可以复用的约定，最后把不同能力组织起来。

先看最左边。Chat 是我们探索想法的起点，比如我想做一个统一入口，帮助用户找到合适的 Agent。这时想法还比较粗，我们可以先和 AI 来回讨论。到了 Structured Prompt，就要把讨论整理成一个清楚的任务，让 AI 知道这轮具体要做什么，最后交付什么。

接下来是 Reusable Instructions。讨论里有些内容确认以后，下次还会继续用，比如 Hub 负责什么、业务 Agent 应该怎样写。这些就可以保存到 README 和 Agent TXT 里面，成为后续工作的共同参照。这些具体内容，我们稍后会提到。

接下来是 Workflow。只有角色说明还不够，我们还要把判断和交接连起来。用户只是浏览时走哪里，缺少材料时停在哪里，方案确认后交给谁，这些都需要在流程里说清楚。

到了 Agent，我们希望它能在规定范围内完成工作。最后是搭建 Agent Hub，把这些职责不同的 Agent 组织到一个入口里面。

这张图是帮助我们理解设计思路的路线。接下来就从最开始的产品问题看起，为什么我们会想做 Agent Hub?

第 4 页 | 为什么要做 Agent Hub

我们先看左边这个情境。现在可能有做 Research 的 Agent，有处理 Data、Documents 和 Code 的 Agents。选择越来越多，但用户拿到任务的时候，通常先知道自己要完成什么，不一定知道应该找哪一个，或者说，哪一个最适合自己的项目。

比如我要核对一份文件，光看 Agent 的名字，很难知道它能检查哪些内容，需要什么材料，有哪些限制。所以我们提出的第一个设计问题，就是怎样从用户的任务出发，帮助他找到合适的能力。

我们希望能够设计这样一个 Agent，返回一个候选清单，标明 Strong fit、Partial fit 和 Limited fit。它的能力是否覆盖这个任务，输入是否匹配，权限是否满足，还有哪些事情做不了，都应该一起说明。

但推荐完以后，我们希望能够继续往下走，先选择 Agent，需要时形成计划，再让用户确认范围，然后在允许的范围内执行，最后返回可以检查的结果。

这样我们就有了一个更具体的设计目标。我们想要设计的 Agent Hub 应该可以把能力发现、适配说明和任务协调接起来。但是，这些工作应该都放在 Hub 里面，还是拆给不同的角色?

第 5 页 | 和 AI 一起确定主体框架

和 AI 一起确定主体框架，我们可以分成四步，分享想法、拆解工作、分配角色，再明确边界。右边的对话是把这个设计讨论整理成了一个示例。

第一步，我们先把想法告诉 AI。我想做一个统一入口，能够推荐 Agent、解释为什么适合，也能帮助用户完成任务。这里先请它帮忙整理系统结构。

AI 就把要支持的工作拆出来了。先理解用户请求，再匹配能力，需要时规划 Workflow，然后执行任务。如果现有能力不够，还要处理这个缺口。

但列出工作以后，还需要我们继续判断。所以接下来追问，哪些属于主 Agent 的工作，哪些值得独立成一个功能 Agent? 特别是规划和执行，要分别由谁负责，各自的边界是什么?

这样也就形成了左边的图。中间的 Hub 面向用户，负责交互和路由。周围四个功能角色分别是 Finder、Creator、Builder 和 Executor，各有一项主要交付。这里的布局展示的是角色关系，没有规定每次都按这个顺序经过四个角色。到这一步，我们有了一个角色框架。但只说 Finder 负责查找、Builder 负责规划，还不能直接指导后面的工作。我们还得把它们之间具体交接什么讲清楚。

第 6 页｜用泳道图核对职责和交接

所以接下来，我们主要看三个问题，Owner、Handoff 和 Boundary。也就是每一步谁负责，接收和交付什么，以及什么情况下应该暂停或返回。

USER 是使用 Agent Hub 的人，HUB 是统一入口，FUNCTIONAL 是刚才那些功能角色，BUSINESS 则是完成具体业务任务的 Agent。我们作为 Designer，是在和 AI 一起设计这套关系。

我们沿着图走一遍。用户先提出 Request，Hub 理解目标，把查找能力的工作交给 Finder。Finder 返回候选和适配理由，再由 Hub 呈现给用户选择。如果任务需要多个步骤，Builder 就把步骤和依赖整理成 Blueprint。这个 Blueprint 先回到 Hub，由 Hub 展示给用户，确认要做哪些事情、范围到哪里。确认以后，Executor 才开始调度业务 Agent。业务 Agent 完成具体工作，返回结构化结果，最后由 Hub 检查并向用户呈现。这样看 Owner，我们知道每一步找谁。看 Handoff，我们就要检查传过去的是不是足够明确。比如 Finder 不能只传一个名字，Builder 也不能只说“接下来执行”，要把候选理由、步骤、输入和输出写出来。再看 Boundary，缺少必要材料时应该暂停，方案还没有确认时也不能直接进入执行。

这一步，AI 帮我们把角色关系画出来、把交接展开，我们再检查这样的分工是否符合需求，哪些边界需要继续补清楚。图里先展示典型路径，接下来我们还会再补不同请求下的分支。

第 7 页｜先请 AI 连出一条主线

角色和交接初步清楚以后，我们可以给 AI 这个具体任务。大家看右边第一句，根据已经确定的角色，把用户提出请求到拿到可审核结果的主线连起来。

于是得到左边这条路径。用户提出请求，Hub 理解和判断，接着查找或核验 Agent，需要时规划 Workflow，再确认、执行和审核。

这里的“需要时”很重要。如果只是一个简单任务，就不一定要专门走一轮复杂规划。我们先保留这条主线，让它足够清楚。

但拿到第一版以后，我们还要继续问。大家看右边后面的追问，如果站在用户的角度，有哪些不同问法，或者缺少哪些输入，会让这条路径发生变化？

比如用户只想看看有哪些 Agent，或者他已经指定了一个 Agent，这两种情况还需要走同一条路线吗？所以这页下面的结论是，主线只是起点。接下来我们要用具体问题，把遗漏的分支找出来。

第 8 页｜把主线对应到具体角色

在检查分支之前，我们先把主线里的负责人对应起来。左边还是刚才那条路径，我们先用几个大家比较熟悉的类比，简单介绍一下 Hub、Finder、Builder 和 Executor 这几个组件。

Hub 有点像 programming lead，先接住请求，判断需要谁参与，把推荐或者方案带回来让用户确认。

Finder 像 macro library search。它根据任务去查 catalog，说明哪些能力合适、有哪些限制、还缺什么输入。它负责推荐，到这里还没有开始执行。

Builder 像 programming planner。假设一个任务有 SDTM、ADaM、TLF 这几个阶段，就需要说明先后顺序、每步输入输出，以及后一步依赖前一步的什么结果。简单任务可以跳过这一步。

Executor 则像 batch-job runner，按照已经确认的步骤调度工作，并报告哪些完成了、哪些失败了、哪些被跳过。某一步失败以后，不能默认把整套任务从头再跑一遍。

接下来，我们就带着这几个分工，以三种用户问法为例来检查流程。

第 9 页｜用户只是想浏览时怎么办

第一种问法很简单，用户说，我想看看现在有哪些 Agent。

大家看左边，这里先从用户问题进入，由 Hub 判断意图。如果只是浏览或者找能力，就交给 Finder 返回概览或候选，回答完就可以结束。如果用户希望我们替他完成任务，才需要继续检查目标、输入和限制，再选择能力、形成计划。

那我们怎样和 AI 把这个区别说清楚？看右边第一轮提问，用户只问有哪些 Agent，Hub 应不应该执行什么？AI 给出的规则是，把浏览和执行分开，Finder 返回概览或推荐。

但这个回答还没有覆盖所有情况，所以我们再追问。如果用户的目标已经很明确，只是还没上传文件呢？比如他说要核对 spec，但材料晚一点才给。

这时可以先讨论适合的能力，说明需要哪些材料，但执行要等必要输入齐全。如果连目标也不清楚，就先问几个有针对性的问题。

这样看，补规则的过程就是从一个用户问法开始，再用边界情况追问，最后把它写回流程。这里补清楚了两件事，浏览在哪里结束，任务执行又要满足什么条件才能继续。

第 10 页｜用户已经指定了 Agent 怎么办

第二种情况，用户直接说，我就想用这个 Agent。

那 Hub 还要不要重新找 Finder，推荐一遍所有候选？我们先看右边的提问，AI 给出的初步设计是，先核验用户指定的 Agent。如果它适合这个任务，而且具备执行条件，就可以跳过重新推荐。

对应左边的分支，先判断用户有没有指定 Agent。有的话，检查它是否存在、能力是否适合目标，以及当前输入和权限是否满足。如果没有指定，或者这个选择无效、不合适，再请 Finder 找替代项。

接着还要追问两个问题。如果选错了怎么办？用户已经选了，是不是就可以直接执行？

所以这里继续补上规则。选择无效时，先解释原因，再提供替代选项。即使选对了，也仍然要确认执行范围，不能把“我选这个”理解成对后面所有操作的授权。

比如用户选好了一个核对文件的 Agent，但必要文件还没有提供，那我们可以确认它适合做这件事，同时把缺失输入列出来，等材料齐全后再继续。

这样就把选择、适配和执行确认分开了，流程也不会因为用户说出了一个 Agent 名字，就跳过后面的检查。

第 11 页｜一个任务需要多个 Agent 怎么办

第三种情况，用户的目标需要几个 Agent 配合完成。

我们先看左边。目标和候选能力确定以后，还要判断这是一个简单任务，还是有多个阶段。如果一个 Agent 就能完成，可以形成简单的单 Agent 方案。如果有多个阶段，就交给 Builder 拆解。

右边第一轮提问，是怎样让几个 Agent 一起完成一个复杂任务。AI 提出，把目标拆成有顺序的步骤，写清每步负责人、输入、输出和依赖。

我们再接着问，谁来规划，谁来执行，它们之间到底交接什么？这一步是在把“配合完成”变成可以检查的约定。

比如前面提到 SDTM、ADaM 和 TLF，不能只把三个名字排在一起。还要说明前一步交付什么，后一步拿什么作为输入，缺少哪项材料时不能继续。

所以最后形成的规则是，Builder 产出 Blueprint，Hub 把方案呈现给用户确认，Executor 再接收已经确认的计划，并在这个范围内调度工作，结果最后回到 Hub。

到这里，我们用三种问法补了浏览、指定 Agent 和多 Agent 协作。接下来再把这些分支放回一张完整流程图，看看有没有接不上的地方。

第 12 页｜把主线、分支和返回路径连起来

这里展示的是项目中的完整流程图，可以打开后放大查看。我们不用逐个念所有节点，重点是沿着箭头检查，不同情况下一步到底去哪里。

右边第一句，就是请 AI 把刚才讨论的主线和用户问题分支合到一起，标出每条分支在哪里回到主线，在哪里等待，在哪里结束。AI 可以帮我们连接正常路径、澄清循环、方案修改和结果审核。如果有一条线不知道怎么接，就先指出缺少什么决定，留给我们判断。

接下来，我们就根据右边最后一句涉及的内容，来看几个例子。输入缺失、方案被拒绝，或者执行失败时，下一步分别在哪里？先看输入缺失。用户说选好了 Agent，但文件晚点给，流程应该停在输入检查，由 Hub 返回缺少的字段。等材料补齐，再继续相关路径。再看方案被拒绝。应该回到修改或澄清，更新方案以后重新呈现，不能从拒绝直接接到执行。如果执行中某一步失败，就要保留失败原因、已经完成的部分，以及后续哪些步骤被跳过，再决定怎样继续。默认从头重跑，可能会重复已经完成的工作。还有前面那条浏览路径，返回概览或候选以后就可以结束。

所以这张图的作用，是帮助我们做设计走查，让我们和 AI 一起检查，确保每种情况都有明确的下一步、返回位置或停止点。

第 13 页｜先整理想法，再按共同文件继续做

到现在，我们已经讨论了不少角色、流程和边界。如果这些内容都留在聊天里，下次继续做，还要重新翻记录，判断哪些已经确认、哪些还只是建议。

所以这页把和 AI 的配合分成两次。第一次，先请 AI 帮我们整理想法。大家看左边，原始内容可能混着角色、写作风格、步骤和例外情况。我们让 AI 把相关内容放在一起，指出缺少的细节，把开放问题单独列出来，再由我们做 Human review。审核以后，才进入中间这组共享文件。写作标准记录风格、格式和限制。README 保存角色、规则和决定。Workflow 和 TODO 则记录步骤、交

接、检查项，以及下一步。比如“执行前需要确认范围”，讨论通过以后，就应该写进相关规则。“这个异常情况到底怎么处理”，如果还没有决定，就继续留在开放问题里。

第二次，是让 AI 根据这些已经确认的文件继续工作。先读当前参照，再按步骤生成或更新文件，最后检查并汇报。大家再看底部这个回环。发现问题以后，我们还要判断，是生成内容没有遵守约定，还是原来的约定就不够清楚。前一种要修正生成内容，后一种需要先补充约定，再更新共同文件。

这样下一轮就有一个明确的起点。下面我们具体看，这些文件分别负责保存什么。

第 14 页｜四类文件分别做什么

这页我们从左到右看四类文件，每一类都有一个主要用途。

第一个是根目录的 README，保存整个系统的设计，包括角色、共同规则和已经接受的决定。想知道 Hub 怎样协调其他角色，先从这里看。

第二个是 Business README，规定业务 Agent 应该怎样写。比如 Name、Description 和 Instruction 分别包含什么，输出和边界怎样描述。新增业务 Agent 时，就按这个格式来。

第三个是 Registry，用来发现能力，有点像 macro library 的 catalog。大家看卡片里列的几个字段，有状态、能力、必需输入和适用范围。比如一个 Agent 可以核对 spec 和 define.xml，那么这两个文件就是需要检查的输入。如果 Scope 只写 ADaM，就不能因为它能做文件核对，直接推断它也支持 SDTM。状态里还会区分 Active、Testing 和 Deprecated。列在 Registry 里，表示我们能查到这项能力。是否已经在某个 study 验证过、当前工具是否具备运行条件，还需要另外检查。

最后是 TODO，记录任务、检查和未解决事项。它帮助我们知道哪些已经做完，哪些还没完成，下一轮从哪里继续。

所以后面和 AI 沟通时，我们可以具体说，先读哪份参照，这轮更新哪些文件。比如新增业务 Agent，要按 Business README 写定义，同时更新 Registry，再把检查和剩余事项记到 TODO。

文件怎样分工、哪些内容成为共同规则，需要我们确认。AI 帮忙整理和更新以后，我们再检查这些文件之间有没有遗漏或矛盾。

第 15 页｜请 AI 整理约定，再检查实际 README

我们先看左边这个请求。把已经确认的角色、Workflow 和规则整理进 README，保留已有决定，同时标出还没解决的问题。这里“已经确认”很关键。如果只是让 AI 总结整段对话，它可能把讨论中的建议也放进去。所以我们要明确，让它区分接受的选择和开放问题。

中间给了一个固定框架。第一组是 Responsibilities，对应 Core Architecture。这里要说明 Hub、Finder、Builder、Executor 和 Creator 分别负责什么。第二组是 Workflow Rules，对应 Task Routing Logic。比如浏览时怎样返回结果，需要推荐时怎样找 Finder，简单和复杂任务分别走什么路径。第三组是 Decision Record。已经接受的选择放进 Decision Log，还没决定的内容留在 Open Questions。下次再看，就能知道哪些可以直接沿用，哪些还需要讨论。

接下来再看右边的实际 README。比如讲到 Finder，我们就检查它的职责有没有对应前面的约定，是否把候选和适配说明写清楚，是否守住了推荐和执行的边界。看 Decisions 时，再检查已经确认的内容有没有保存，未决问题有没有被误写成结论。这样我们检查的是讨论有没有准确落进文件，之后才能继续拿它指导 AI。

第 16 页｜用共同标准检查一个具体 Agent 定义

项目层面的 README 有了以后，每个业务 Agent 也需要按照同一套标准来写。我们先看左边的 business-subagents/README.md。这里简单地分成三个部分。

第一部分是 Definition Format。Name 里要有中英文名称，Description 要在 800 字符以内，说明这个 Agent 做什么。Instruction 再展开角色、触发条件、输入、输出、规则和限制。

第二部分是 Shared Boundaries。结果要保持结构化，交回 Hub 或上游功能 Agent。缺少必要输入时，返回 needs_clarification，用 missing_fields 说明缺什么，有假设时也要明确记录。

第三部分是 Consistency Checks。TXT 里怎样描述，Registry 里就要保持一致，不能定义里有限制，catalog 却把它写成什么都能做。

接下来我们对照右边的 Presentation Rehearsal Coach Agent，也就是演讲排练反馈助手。先看名称和 Description，它说明会根据提供的演讲内容，检查表达清晰度、结构、听众适配和预计时间，也可以提供明确标注为模拟的听众问题。

再往下看 Instruction。这里规定它把结构化结果返回上游。输入要求里，需要实际讲稿、转录或练习回答，以及听众和表达目标。如果缺少必要内容，就列出最少的 missing_fields，先返回澄清状态。

这样就能把左边的共同标准和右边的具体定义对应起来。最后还要核对 Registry metadata，确认能力、范围和状态这些信息一致。新增下一个 Agent 时，也沿用同样的检查方式。

这里可以先让 AI 按标准生成，再逐项检查有没有缺字段、有没有描述不一致的地方。我们重点审核它对业务能力和限制的理解，确认这些内容确实符合这个 Agent 的定位。

第 17 页｜一次只推进一个约定好的步骤

文件标准都有了，接下来我们怎样请 AI 继续做？大家看上面这个请求，先读当前 README 和 TODO，提出实现顺序。这一轮只完善 Hub 和 Finder 的定义，再汇报检查结果和待决定的问题。

下面把这个过程拆成四步。第一步，读参照，知道当前角色、格式和开放任务是什么。第二步，限定本轮范围。比如这次只改 Hub 和 Finder，就把涉及的文件和输出说清楚，让这一轮有一个可以检查的交付。第三步，生成、检查、修订。拿到初稿以后，回到已经接受的规则，看职责有没有变，交接有没有缺，输出格式是否一致。第四步，更新 TODO。完成了什么，还有什么没解决，下一步做什么，都要留下来。

每轮结束都要汇报修改文件、做过的检查和开放问题。如果下一步需要改变已经同意的职责、规则，或者扩大范围，就先把这个变化提出来。这样下一轮继续时，我们就能接着已经完成的部分往下做。

第 18 页｜继续工作前，先恢复共同约定

但隔了一段时间再回来，或者换一个 conversation，我们经常会遇到左边这三个问题。前面的决定没有带过来，已经确认的角色又被重新设计，或者本来只改一个小地方，最后连无关分支也一起改了。所以继续之前，我们先做右边这四步，读参照、恢复约定、定位修改，再在范围内继续。

先看第一句给 AI 的要求。读取当前 README、相关 Workflow、Agent TXT 和 TODO，先总结已经接受的角色、关键规则、完成内容和开放事项。我们先确认它恢复的理解是对的。接下来，让 AI 定位这次请求影响哪条分支，涉及哪些文件，准备怎样改。如果参照之间有冲突，或者缺少一个关键决定，就先指出来。如果这次带来的是一个新想法，也先按这个顺序看。我们可以请 AI 对照现有 README、流程图和 Agent 指令，列出它影响哪些角色、分支和交接，和已有规则有没有冲突，再提出可选的处理方式。我们先看影响，再决定是否修改。

比如我们可能会想到，用户已经指定 Agent 时，能不能少问一轮确认？这时就要请 AI 找出当前关于执行确认的约定，说明减少的是重复询问，还是会改变原来确认执行范围的规则。还要看目标、材料和范围是否已经清楚，用户是否已经明确同意执行。AI 帮我们把这些区别和影响列出来，再由

我们判断，这个想法是否真的解决了问题，原来的边界是否仍然成立。决定保留原规则，就把原因记录下来。暂时判断不了，就放进开放问题。确认要改以后，再让 AI 按接受的方案更新相关文件。

最后一句再明确范围，只更新已经约定的部分，保留无关分支。结束以后，汇报改了什么、检查了什么、还有什么问题。如果要改变已经接受的角色或规则，就先说明原因，让我们判断。这和九月份讲的 Context 是连着的。我们把当前有效的信息恢复清楚，再让 AI 接着工作。

第 19 页｜拿一个具体请求检查修改

上一页确认要修改以后，还要把同一项决定同步到相关文件。可以先让 AI 列出需要更新的位置，比如 README 里的规则、流程图上的分支，以及 Hub 或相关 Agent 的指令。如果涉及能力、输入或状态的变化，也要核对 Registry，再把完成内容和待验证事项记进 TODO。我们审核这些更新是否表达了同一个意思。

那文件改完以后，我们具体怎么检查？这页就拿一条请求走一遍。用户说，用这个 Agent，文件晚点给。

先看上面的四步，选一条请求，沿分支走，对照文件，再修正不一致。我们沿中间这条路径看。第一步，核验这个 Agent 是否适合用户目标。假设适合，接着检查必要输入。现在文件还没给，所以流程应当暂停，由 Hub 返回需要补充的字段。然后看右边，要同时对照 README、Hub 和相关 Agent TXT。README 和 Hub 的路由有没有保留输入检查？Agent TXT 有没有明确缺少哪些输入要暂停，以及怎样返回 missing_fields？比如一处写“选好 Agent 就执行”，另一处却写“缺少必要输入要先澄清”，这两处就需要对齐。我们给 AI 的反馈可以很具体，请指出冲突条款，说明怎样修改，再更新受影响的文件。修完以后，还要用同一条请求重新走一遍，看它是否会停在正确的位置，是否返回了需要的材料。

如果这轮修改来自刚才那个减少重复确认的想法，还要回头检查，这条缺少文件的请求是否仍然会暂停。我们可以请 AI 帮忙找受影响的情况、逐条走查，再由我们审核结果，确认新规则解决了原来的问题，也没有漏掉仍然需要保留的检查。

这里做的是文件和设计规则的走查。要判断实际调用是否也按这个规则运行，就需要下一页这些不同层面的证据。

第 20 页｜分清定义、运行和结果分别完成到哪里

大家先看上面这句话，文件都更新了，还需要进一步确认功能完成到了哪一层。

第一层是 Definition。角色和规则是否一致，TXT 和 Registry 是否对得上。前面我们做的文件检查和场景走查，主要在这一层。

第二层是 Runtime。工具和权限是否配置好，实际有没有发生 Agent 调用，需要有运行记录来支持。

第三层是 Results。运行以后返回了什么，这个输出有没有证据，是否符合验收要求，还需要继续审核。

下面是项目里一个实际的修正，10 月 15 日的 TODO 专门澄清了 Active 的含义。它表示 TXT 定义完整，可以被发现。至于部署和运行验证，还需要单独的证据。

所以汇报进度时，我们可以明确说，定义已经检查，运行证据待补，结果还没有审核。这样大家就知道下一步缺什么，也不会把几个阶段混在一起。

第 21 页｜每轮迭代，人和 AI 分别做什么

前面的泳道图讲的是 Hub 内部的角色交接。这一页我们换个角度，回看 Designer 和 AI 怎样一起完成这些设计。

先看 Define。由我们把目标、角色边界和要求交代给 AI。比如要做统一入口，要区分推荐、规划和执行，这些都是需要我们明确的方向。接下来是 Develop。AI 帮忙整理想法、提出方案，并生成可以拿来检查的文件。前面看到的角色框架、Workflow 初稿和 README 草稿，都属于这样的产出。然后是 Review。我们检查业务含义、依据和冲突，再给具体反馈。比如用户只是浏览，为什么会进入执行？材料缺失以后，这条分支应该停在哪里？这样反馈以后，AI 才知道需要改哪条规则。最后是 Record。把已经接受的决定、更新的文件和下一步保存下来，再由我们核对。还在讨论中的建议，要继续作为开放问题保留。

人的判断指导每轮迭代，AI 帮助整理、展开和记录工作。下一轮继续时，就从这些确认过的内容往前走。

第 22 页｜把沟通方式整理成一个可复用模板

最后，我们把前面的做法整理成一个可以带走的模板。左边有五个要素，Goal、Context、Scope、Constraints 和 Acceptance。右边把它们放进了一次完整请求。

先看 Goal，说明我们想实现什么。这里就是设计 Agent Hub，提供一个发现和协调业务 Agent 的统一入口。

Context 说明 AI 应该根据什么工作。使用我们的需求和现有项目文件，先总结已经接受的决定和开放问题。这一步就和前面恢复上下文对应起来了。

Scope 则限定这一轮要交付什么。比如把常见用户请求整理成 Workflow，定义 Hub、功能 Agent 和业务 Agent 的职责，提出最小文件结构，再写一版 README。README 里要覆盖愿景、架构、路由、定义标准、决定和开放问题。

Constraints 把必须遵守的边界写出来。推荐、规划和执行分开，执行前需要确认。缺少决定就标出来，工具、部署和测试结果都要依据已有材料描述。

最后是 Acceptance，也就是我们拿什么判断这轮做完了。这里要求流程图、文件职责和 README 草稿，还要用有代表性的用户问题走一遍，列出冲突、假设和下一步。

大家看底部这三步。先填清五个要素，再限定这一轮只做 Workflow 和 README 初稿，最后拿一条具体请求来检查负责人、缺失输入、交接和停止点。

换成自己的项目，也可以按这个方式写。先说目标和现有依据，再说这轮范围、要守住的规则，以及最后怎么检查。

第 23 页｜把设计变成可以继续检查和维护的共同文件

回到今天最开始的问题，我们怎样和 AI 一起，把一个产品想法逐步做成 Agent 框架?

这页收成三件事。先设计逻辑，把角色、判断和交接讲清楚。再保留共同参照，把接受的规则写进 README、流程图和 Agent 指令。之后每次修改，都检查受影响的文件，也分清定义、运行和结果分别验证到了哪里。

放回 Agent Hub 这个 case，我们先提出统一入口的目标，再和 AI 拆角色、连主线，用用户问题补分支，然后把这些约定保存下来，指导后面的生成和检查。

后面有了新想法，也先让 AI 分析它与现有规则的冲突和影响，再由我们决定是否接受。确认修改后，同步相关文件、更新 TODO，再用具体请求检查。AI 协助梳理、生成和检查，我们负责判断需求、确认规则和审核结果。这样项目才能在已有设计上继续迭代。

所以最后这几句话，也是我们可以带回自己工作里的做法。把想法整理成 Context，把要求保存成标准，再用文件指导 AI 实现和检查。

今天主要讲到设计和定义。下一次可以继续往运行层看，怎样配置 Knowledge、接入 Tools，完成实际调用，再验证输出。谢谢大家。